BSM 451 – AĞ PROGRAMLAMA VİZE ÇALIŞMA REHBERİ

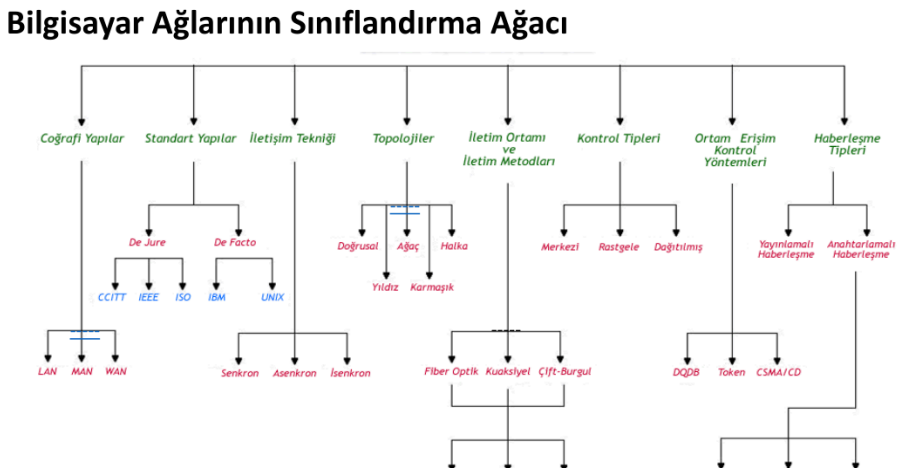
Bölüm 1: Hafta 1 (OSI), Hafta 2 (PCAP, Ethernet & ARP), Hafta 3 (IPv4, Fragmantasyon, IPv6 & ICMP) ve 4 Adet Libpcap C Kodu Analizi

Bu doküman, **Dr. Öğr. Üyesi Musa BALTA**'nın ders sunumlarındaki tüm şemaları, sayısal örnekleri ve `Katman2-3 Kodlar.rar` içindeki 4 adet C kodunu (`1-arpgonder.c`, `2-icmpgonder.c`, `ping_client.c`, `ping_server.c`) **PDF Sayfa Numarası, Slayt Numarası ve Kod Satır Numaralarıyla** birebir eşleştirerek sınav odaklı olarak özetlemektedir.

1. DERS: HAFTA 1 – OSI REFERANS MODELİ VE KATMANSAL YAPI

Kaynak Dosya: Hafta-1-OSI Referans Modeli ve Katmasal Yap .pdf (Toplam 4 PDF Sayfası / 8 Slayt)

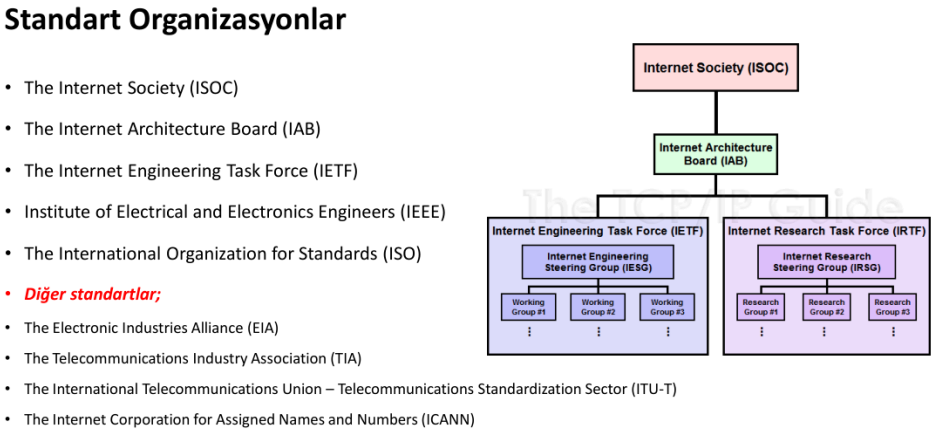
1.1. Bilgisayar Ağlarının Sınıflandırma Ağacı (PDF Sayfa 2, Slayt 3)



Görsel: Hafta-1 PDF Sayfa 2, Slayt 3 – Bilgisayar Ağlarının Sınıflandırma Ağacı

- **1) Coğrafi Yapılar:** `LAN` (Yerel Alan Ağı), `MAN` (Metropol Alan Ağı), `WAN` (Geniş Alan Ağı).
- **2) Standart Yapılar (Sınavda De Jure / De Facto ayrımına dikkat!):**
  - **De Jure (Yasal / Resmi Kurum Standartları):** `CCITT`, `IEEE`, `ISO`.
  - **De Facto (Fiili / Pratikte Kabul Gören Standartlar):** `IBM`, `UNIX`.
- **3) İletişim Tekniği:** `Senkron`, `Asenkron`, `İsenkron` (Sabit zaman aralıklı).
- **4) Topolojiler:** `Doğrusal` (Bus), `Yıldız` (Star), `Ağaç` (Tree), `Karmaşık` (Mesh), `Halka` (Ring).
- **5) İletim Ortamı ve İletim Metodları:** Ortamlar: `Fiber Optik`, `Koaksiyel`, `Çift-Burgulu`. Metodlar: `Geniş Band` (Broadband), `Taşıyıcı Band` (Carrierband), `Temel Band` (Baseband).
- **6) Kontrol Tipleri:** `Merkezi`, `Rastgele`, `Dağıtılmış`.
- **7) Ortam Erişim Kontrol Yöntemleri:** `DQDB`, `Token` (Jeton), `CSMA/CD`.
- **8) Haberleşme Tipleri:** `Yayınlamalı Haberleşme` (Broadcast) ve `Anahtarlamalı Haberleşme` (**Devre Anahtarlamalı**, **Mesaj Anahtarlamalı**, **Paket Anahtarlamalı**).

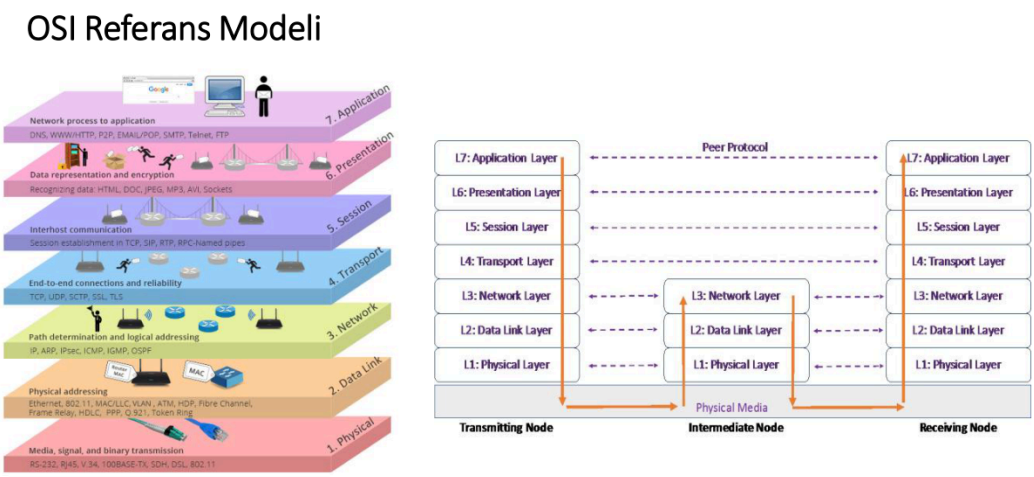
1.2. Standart Organizasyonlar Hiyerarşisi (PDF Sayfa 2, Slayt 4)



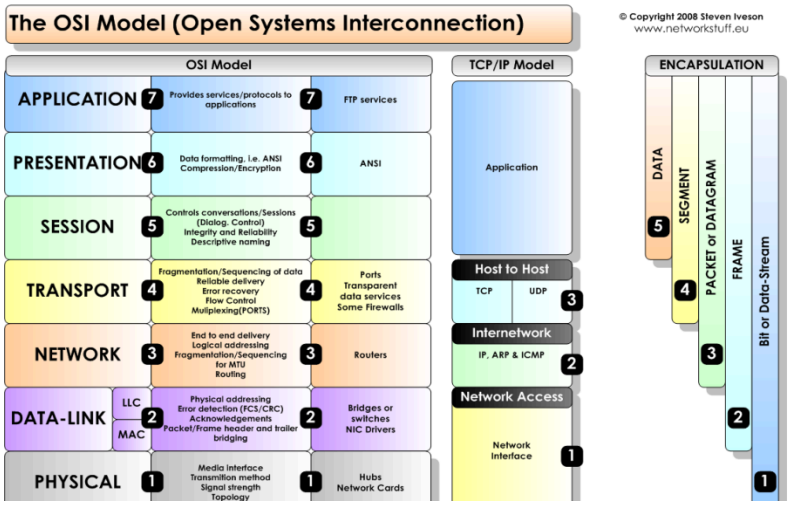
Görsel: Hafta-1 PDF Sayfa 2, Slayt 4 – Standart Organizasyonlar ve ISOC Hiyerarşisi

- **Şemadaki Hiyerarşi:** En üstte **Internet Society (ISOC)** → altında **Internet Architecture Board (IAB)** yer alır. IAB iki ana kola ayrılır:
  1. **Internet Engineering Task Force (IETF)** → **IESG** (Internet Engineering Steering Group) → *Working Group #1, #2, #3...*
  2. **Internet Research Task Force (IRTF)** → **IRSG** (Internet Research Steering Group) → *Research Group #1, #2, #3...*
- **Diğer Kuruluşlar:** `IEEE`, `ISO`, `EIA`, `TIA`, `ITU-T`, `ICANN` (Alan adları ve sayılar), `IANA` (IP ve Port numaraları tahsisi).

1.3. OSI Referans Modeli, Kapsülleme ve Cihazlar (PDF Sayfa 3, Slayt 5–6 & PDF Sayfa 4, Slayt 7)



Görsel: Hafta-1 PDF Sayfa 3, Slayt 5 – OSI 7 Katman ve Ara Düğüm (Intermediate Node) Yapısı



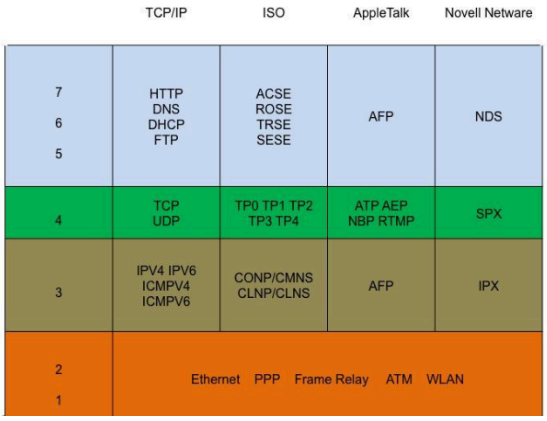
Görsel: Hafta-1 PDF Sayfa 3, Slayt 6 – OSI Modeli, Cihazlar, TCP/IP Karşılığı ve Kapsülleme (Encapsulation)

SINAV DETAYI (Slayt 5 ve Slayt 6):

- **Intermediate Node (Ara Düğüm / Router):** Slayt 5'in sağındaki şemada görüldüğü üzere uç düğümler (Transmitting ve Receiving Node) 7 katmanın tamamını (L1–L7) çalıştırırken, aradaki yönlendiriciler (Intermediate Node) **yalnızca ilk 3 katmanı (L1: Physical, L2: Data Link, L3: Network)** çalıştırır!
- **Kapsülleme (Encapsulation) Sırası (Slayt 6 Sağ Sütun):** Üst katmanlarda **DATA** (L7–L5) → Taşıma katmanında (L4) **SEGMENT** → Ağ katmanında (L3) **PACKET veya DATAGRAM** → Veri Bağı katmanında (L2) **FRAME (Çerçeve)** → Fiziksel katmanda (L1) **Bit veya Data-Stream** adını alır.

No	OSI Katmanı	Temel Görev ve Protokoller (Slayt 5–7)	Arabağlantı Cihazı (Slayt 6)	TCP/IP Modeli	Veri Birimi (PDU)
7	**Application**	Uygulama servisleri (`HTTP, DNS, DHCP, FTP, Telnet, SMTP`)	FTP services	Application	**DATA**
6	**Presentation**	Veri formatlama, sıkıştırma, şifreleme (`ANSI, HTML, JPEG, SSL/TLS`)	ANSI	Application	**DATA**
5	**Session**	Oturum kontrolü, diyalog yönetimi (`NetBIOS, RPC, SQL`)	—	Application	**DATA**
4	**Transport**	Uçtan uca güvenilirlik, akış/hata kontrolü, port çoklama (`TCP, UDP`)	Ports, Bazı Firewall'lar	**Host to Host**	**SEGMENT**
3	**Network**	Mantıksal adresleme (IP), yönlendirme, parçalama (`IP, ARP, ICMP, IGMP, OSPF`)	**Routers (Yönlendiriciler)**	**Internetwork**	**PACKET / DATAGRAM**
2	**Data-Link (LLC+MAC)**	Fiziksel adresleme (MAC), hata tespiti (FCS/CRC) (`Ethernet, PPP, ATM`)	**Bridges / Switches, NIC**	**Network Access**	**FRAME**
1	**Physical**	Kablo, sinyal gücü, ikili (binary) bit iletimi (`RS-232, RJ45, 100BASE-TX`)	**Hubs, Network Cards**	**Network Access**	**Bit / Data-Stream**

Farklı Standartların Karşılaştırımı



Görsel: Hafta-1 PDF Sayfa 4, Slayt 7 – Farklı Standartların (TCP/IP, ISO, AppleTalk, Novell Netware) Karşılaştırımı

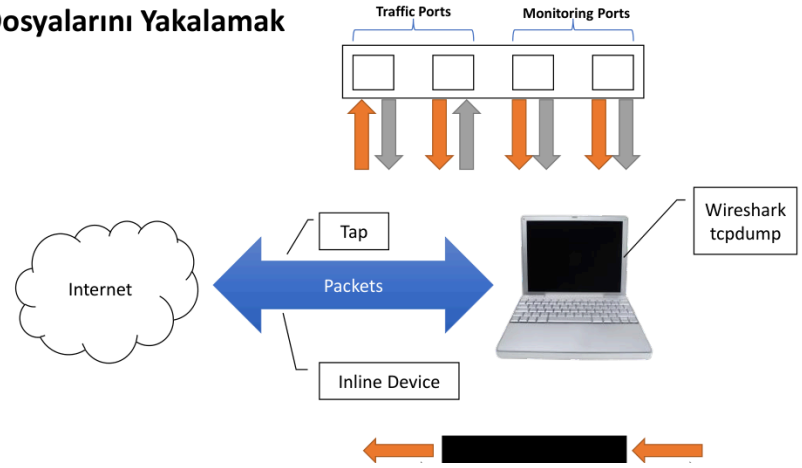
2. DERS: HAFTA 2 (BÖLÜM 1) – PCAP VE LIBPCAP KÜTÜPHANESİ

Kaynak Dosya: Hafta-2 Pcap.pdf (Toplam 9 PDF Sayfası / 17 Slayt)

2.1. PCAP Kavramı ve Paket Yakalama Yöntemleri (PDF Sayfa 2, Slayt 3–4)

- **Slayt 3 (PCAP == Packet Capture):** Ağ aktivitelerinin **Katman 2'den Katman 7'ye kadar** tüm kaydıdır. En yaygın açık kaynak kütüphanesi `libpcap` (*nix ve Windows uyumlu; Windows karşılığı `WinPcap`).
- **Kullanım Alanları (Slayt 3):** Ağ trafiğini dinlemek, bant genişliği kontrolü, pasif DNS çözümlemesi, saldırı girişimlerini görüntüleme, araştırma testleri.
- **Slayt 4 (PCAP Dosyalarını Yakalamak):** Trafik yakalamanın 3 topolojik yolu: **1) Switch Monitoring Ports (Port Mirroring)**, **2) Network Tap**, **3) Inline Device**.

PCAP Dosyalarını Yakalamak



Görsel: Hafta-2 Pcap PDF Sayfa 2, Slayt 4 – PCAP Dosyalarını Yakalama Yöntemleri

2.2. Ham PCAP Depolama, Dosya Formatı ve Overhead Hesapları (PDF Sayfa 3–4, Slayt 5–7)

Ham PCAP Depolama

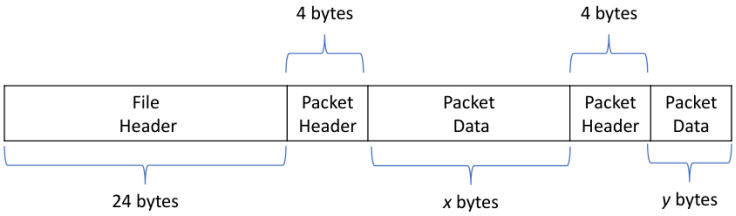
$$1\text{gbps} \times 3600 \times 24 = 86400 \text{ gigabits}$$
$$86400 \div 8 = 10800 \text{ gigabytes}$$

$$1\text{gbps} \div (64\text{bytes} \times 8\text{bits}) =$$
$$10^9 \text{ bits/s} \div 512\text{bits} = 1953125\text{pps}$$

$$N\text{pps} \times H_{\text{PCAP}} = NH\text{Bps}$$

Görsel: Hafta-2 Pcap PDF Sayfa 3, Slayt 5 – Ham PCAP Depolama ve PPS Formülleri

libpcap Format



Görsel: Hafta-2 Pcap PDF Sayfa 3, Slayt 6 – libpcap Dosya Formatı (24B File Header + 4B Packet Header)

libpcap Overhead

Avg. Packet Size (bytes)	Packets Per Second	Overhead (MB/s)	Overhead (GB / day)
64	1,953,125	7.45	628.64
1500	83,333	0.32	26.82
7981	15,662	0.06	5.04
9000	13,888	0.05	4.47

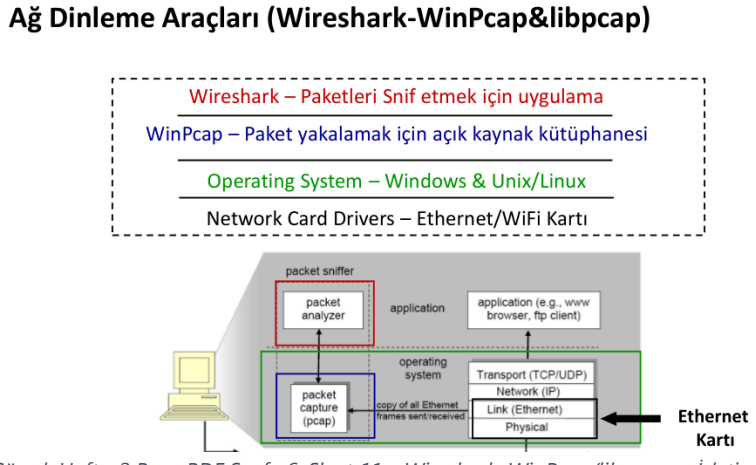
Görsel: Hafta-2 Pcap PDF Sayfa 4, Slayt 7 – Paket Boyutuna Göre libpcap Overhead Tablosu

SAYISAL SINAV SORUSU REHBERİ (Slayt 5, 6 ve 7 Hesaplamaları):

- **1) 1 Gbps Hattın 1 Günlük (24 Saat) Toplam Ham Veri Boyutu (Slayt 5):**
  `1 Gbps × 3600 sn × 24 saat = 86,400 Gigabits` → Byte'a çevirmek için 8'e böleriz: `86,400 / 8 = 10,800 Gigabytes (GB/gün)`.
- **2) Saniyedeki Paket Sayısı (PPS - Packets Per Second) Hesabı (Slayt 5):**
  Ortalama paket boyutu **64 byte** ise bit cinsinden boyutu: `64 bytes × 8 bits = 512 bits`.
  `PPS = 1 Gbps / 512 bits = 1,000,000,000 bits/s / 512 bits = 1,953,125 pps` (Saniyede ~1.95 milyon paket).
- **3) libpcap Dosya Formatı ve Başlık Ek Yükü (Overhead) Hesabı (Slayt 6 & 7):**
  Slayt 6'da görüldüğü gibi dosyanın en başında **1 adet `File Header` (24 bytes)**, ardından yakalanan **her paket için `Packet Header` (Slaytta 4 bytes)** + `Packet Data (x bytes)` bulunur.
  Saniyedeki başlık yükü: `N pps × H_PCAP = N × 4 Bytes/s`.
  Slayt 7 Doğrulaması (64 Byte için): `1,953,125 pps × 4 byte = 7,812,500 Byte/s ÷ (1024 × 1024) = 7.45 MB/s` (Günde `628.64 GB` sadece başlık yükü!). Paket boyutu büyüdükçe (örn. 1500B veya 9000B) saniyedeki paket sayısı (pps) düştüğü için overhead de `0.32 MB/s` ve `0.05 MB/s` seviyesine düşer.

2.3. Ağ Dinleme Araçları (Sniffer) ve Wireshark Mimarisi (PDF Sayfa 4–7, Slayt 8–13)

- **Slayt 8–9 (Sniffer Kavramı):** Ağ trafiğini **pasif izleme (sezme)** veya **aktif (atak yapma)** amacıyla kullanılan donanım/yazılım araçlarıdır. Saldırganlar şifresiz protokollerden (`FTP, POP3, IMAP, SMTP, rlogin, HTTP, VoIP`) pasif veri toplamak için kullanırken, sistem yöneticileri performans ve saldırı tespiti için kullanır.
- **Slayt 10 (Wireshark Hakkında Kesin Bilinmesi Gerekenler):**
  - **Gerald Combs** tarafından **1998**'de **Ethereal** adıyla başlatılmış, **Haziran 2006**'da **Wireshark** adını almıştır.
  - 750'nin üzerinde protokol destekler ve kapsüllemeli görünüm sunar.
  - **Çok Kritik 2 Sınav Maddesi:** 1) "*Paket sniffer uygulamasıdır, dolayısıyla **bir ağın haritasını çıkartmak için kullanılamaz**.*" 2) "***Pasif bir izleme aracıdır, dolayısıyla ağ verisi üretmez**.*"
- **Slayt 11 & 12 (Mimari ve Topolojideki Yeri):** Alttan üste: `Ethernet/WiFi Kartı` → `Network Card Drivers` → `Operating System` → `WinPcap / libpcap` → `Wireshark`. **Hub** tüm trafiği her porta yaydığı için sniffer tüm ağ trafiğini görür; **Switch** ise çerçeveleri sadece hedef MAC portuna ilettiği için sniffer normal şartlarda diğer portların trafiğini göremez.



Görsel: Hafta-2 Pcap PDF Sayfa 6, Slayt 11 – Wireshark, WinPcap/libpcap ve İşletim Sistemi Mimarisi

2.4. Libpcap C Fonksiyonları ve Syntax Yapısı (PDF Sayfa 7–8, Slayt 14–16)

Libpcap-Cihaz Koklaması

```
pcap_t *pcap_open_live(char *device, int snaplen, int promisc, int to_ms,
char *ebuf)
```

```
#include <pcap.h>
    ...
    pcap_t *handle;

    handle = pcap_open_live(dev, BUFSIZ, 1, 1000, errbuf);
    if (handle == NULL) {
        fprintf(stderr, "Couldn't open device %s: %s\n", dev, errbuf);
        return(2);
    }
```

Görsel: Hafta-2 Pcap PDF Sayfa 8, Slayt 15 – pcap_open_live Prototipi ve Kullanımı

Libpcap-Paket Filtreleme (Örnek Kod)

```
#include <pcap.h>
...
pcap_t *handle;         /* Session handle */
char dev[] = "rl0";        /* Device to sniff on */
char errbuf[PCAP_ERRBUF_SIZE]; /* Error string */
struct bpf_program fp;     /* The compiled filter expression */
char filter_exp[] = "port 23"; /* The filter expression */
bpf_u_int32 mask;       /* The netmask of our sniffing device */
bpf_u_int32 net;        /* The IP of our sniffing device */

if (pcap_lookupnet(dev, &net, &mask, errbuf) == -1) {
    fprintf(stderr, "Can't get netmask for device %s\n", dev);
    net = 0;
    mask = 0;
}
handle = pcap_open_live(dev, BUFSIZ, 1, 1000, errbuf);
if (handle == NULL) {
    fprintf(stderr, "Couldn't open device %s: %s\n", dev, errbuf);
    return(2);
}
if (pcap_compile(handle, &fp, filter_exp, 0, net) == -1) {
    fprintf(stderr, "Couldn't parse filter %s: %s\n", filter_exp, pcap_geterr(handle));
    return(2);
}
```

Görsel: Hafta-2 Pcap PDF Sayfa 8, Slayt 16 – Libpcap Paket Filtreleme Örnek Kodu

Libpcap Fonksiyonu	Slayt Yeri	C Kodlarındaki Satır Yeri	Parametreleri, Dönüş Değeri ve Görevi
`pcap_lookupdev(errbuf)`	Slayt 14 (Sayfa 7)	`1-arpgonder.c` Satır 14 `ping_server.c` Satır 37	Sistemdeki varsayılan aktif ağ kartı adını (örn. `"eth0"`) döndürür. Bulamazsa `NULL` döner ve hatayı `errbuf` (`PCAP_ERRBUF_SIZE`) içine yazar.
`pcap_lookupnet(dev, &net, &mask, errbuf)`	Slayt 16 (Sayfa 8)	`ping_client.c` Satır 59 `ping_server.c` Satır 47	Ağ kartının IP ağ adresini (`net`) ve alt ağ maskesini (`mask`) `bpf_u_int32` tipinde alır. Başarısızsa `-1` döner.
`pcap_open_live(dev, snaplen, promisc, to_ms, errbuf)`	Slayt 15 (Sayfa 8)	`1-arpgonder.c` Satır 22 `ping_server.c` Satır 54	Cihazı paket yakalama/gönderme için açar, `pcap_t*` oturum işaretçisi döner (Hata: `NULL`). `snaplen`: yakalanacak max byte (`BUFSIZ` veya `100`), `promisc`: `1` ise promiscuous mod (tüm paketleri yakala), `to_ms`: okuma zaman aşımı (`1000` ms).
`pcap_compile(handle, &fp, filter_exp, 0, net)`	Slayt 16 (Sayfa 8)	`ping_client.c` Satır 80 `ping_server.c` Satır 67	Metin halindeki filtre ifadesini (Slayt 16'da `"port 23"`, ping kodlarında `"icmp && dst <IP>"`) derleyip `struct bpf_program fp` içine kaydeder (Hata: `-1`).

`pcap_setfilter(handle, &fp)`	Slayt 16 (Sayfa 8)	`ping_client.c` Satır 85 `ping_server.c` Satır 72	Derlenmiş olan `fp` filtresini açık olan `handle` oturumuna uygular (Hata: `-1`).

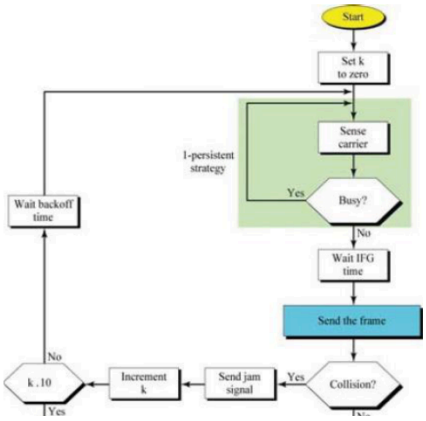
3. DERS: HAFTA 2 (BÖLÜM 2) – KATMAN 2 PROTOKOLLERİ (ETHERNET & ARP) VE 1-arpgonder.c

Kaynak Dosyalar: Hafta-2 Katman 2 Protokolleri.pdf (8 PDF Sayfası / 15 Slayt) & 0-Katman2-3 Kodlar/1-arpgonder.c (Satır 1–108)

3.1. Ethernet Protokolü (IEEE 802.3) ve CSMA/CD Backoff Algoritması (PDF Sayfa 2–3, Slayt 3–5)

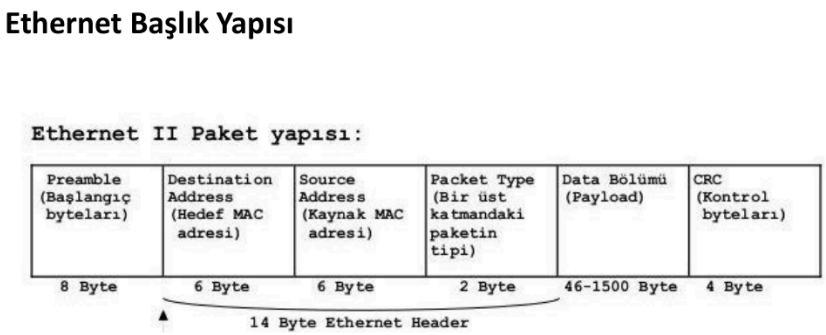
- **Slayt 3 (Ethernet):** En yaygın LAN veri bağı protokolü **IEEE 802.3 Ethernet**'tir (1960'lardaki radyo tabanlı **Alohanet**'in uzantısıdır). Tercih sebepleri: Kurulum kolaylığı, güvenilirlik, ölçeklenebilirlik, çoklu kullanım uyumu.
- **Slayt 4 (CSMA/CD):**
  - **Carrier Sense (Taşıyıcı Sezme):** Kabloda o an aktarım olup olmadığını kontrol eder.
  - **Multiple Access (Çoklu Erişim):** İletim hattındaki tüm düğümlerin aynı hakka sahip olduğunu belirtir.
  - **Collision Detection (Çarpışmayı Sezme):** İki düğümün aynı anda veri göndermesiyle oluşan çarpışmayı algılar.
- **Slayt 5 (Ethernet'te Veri Gönderme / Backoff Akış Şeması):**
  1. `Start` → `Set k to zero` (Deneme sayacı `k = 0` yapılır).
  2. `Sense carrier` (Hattı dinle — *1-persistent strategy*: hat meşgulse boşalana kadar sürekli dinle).
  3. Hat boşsa (`Busy? No`) → `Wait IFG time` (Interframe Gap süresi kadar bekle) → `Send the frame` (Çerçeveyi gönder).
  4. `Collision?` (Çarpışma oldu mu?): Hayır ise `Success`. Evet ise `Send jam signal` (diğer düğümleri uyarmak için bozucu sinyal gönder) → `Increment k` (`k = k + 1`).
  5. Deneme sayısı kontrolü (Slayttaki şemada `k < 10`): Limit aşılmadıysa `Wait backoff time` kadar rastgele bekle ve başa dön; limit aşıldıysa `Abort` (iptal et).

Ethernet'de Veri Gönderme



Görsel: Hafta-2 Katman 2 PDF Sayfa 3, Slayt 5 – Ethernet'te Veri Gönderme (CSMA/CD ve Backoff) Akış Şeması

3.2. Ethernet II Başlık Yapısı (14 Byte) ve İletişim Şekilleri (PDF Sayfa 3–4, Slayt 6–7)



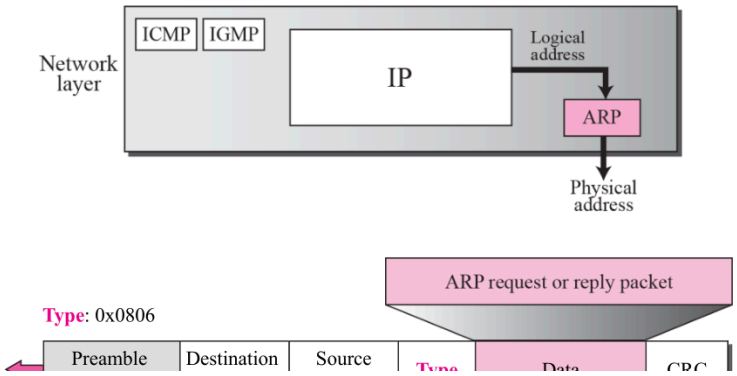
Görsel: Hafta-2 Katman 2 PDF Sayfa 3, Slayt 6 – Ethernet II Paket Yapısı ve 14 Byte Ethernet Header

- **Slayt 6 (Ethernet II Başlığı = 14 Byte):**
  - `Destination Address (Hedef MAC)`: **6 Byte** (`packet[0..5]`)
  - `Source Address (Kaynak MAC)`: **6 Byte** (`packet[6..11]`)
  - `Packet Type (Üst katman protokol tipi)`: **2 Byte** (`packet[12..13]` → ARP için `0x0806`, IPv4 için `0x0800`)
  - **Toplam Ethernet Başlığı:** `6 + 6 + 2 = 14 Byte`! (Baştaki 8 Byte `Preamble` ve sondaki 4 Byte `CRC` ağ kartı donanımı tarafından kabloda eklenir; yazılımdaki 14 byte başlığa dahil değildir).
  - `Data Bölümü (Payload)`: **46 – 1500 Byte** arasındadır.
- **Slayt 7 (Ethernet İletişim Şekilleri): 1) Unicast:** Tek bir MAC adresine (Örn: `00-07-E9-42-AC-28`). **2) Multicast:** Bir gruba; hedef MAC `01-00-5E` ile başlar (Örn: `01-00-5E-00-00-01`). **3) Broadcast:** Ağdaki tüm cihazlara; hedef MAC `FF-FF-FF-FF-FF-FF`.

3.3. ARP (Address Resolution Protocol) Yapısı ve Senaryoları (PDF Sayfa 4–7, Slayt 8–14)

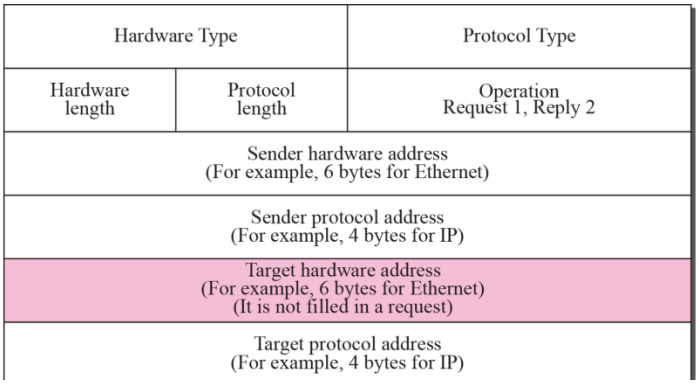
- **Slayt 8–10 (ARP Görevi):** Paketin teslimi için **Mantıksal (IP)** ve **Fiziksel (MAC)** olmak üzere iki düzey adresleme gerekir. ARP, IP protokolünden mantıksal (IP) adresi alır, karşılık gelen fiziksel (MAC) adresle eşler (statik veya dinamik haritalama) ve veri bağı katmanına iletir.
- **Slayt 9 (ARP İşleyişi): ARP Request** tüm yerel ağa yayınlanır (**Broadcast**: Ethernet hedef MAC `FF:FF:FF:FF:FF:FF`), **ARP Reply** ise sadece soran düğüme gönderilir (**Unicast**).

ARP Protokolünün OSI Referans Modelindeki Karşılığı



Görsel: Hafta-2 Katman 2 PDF Sayfa 6, Slayt 11 – ARP Paketinin Ethernet Data Kısmına Kapsüllenmesi (Type: 0x0806)

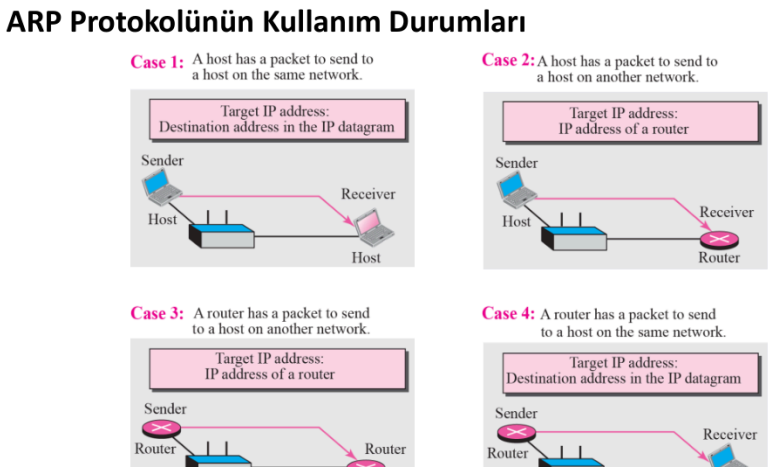
ARP Protokolünün Paket Yapısı



Görsel: Hafta-2 Katman 2 PDF Sayfa 6, Slayt 12 – 28 Byte ARP Paket Yapısı

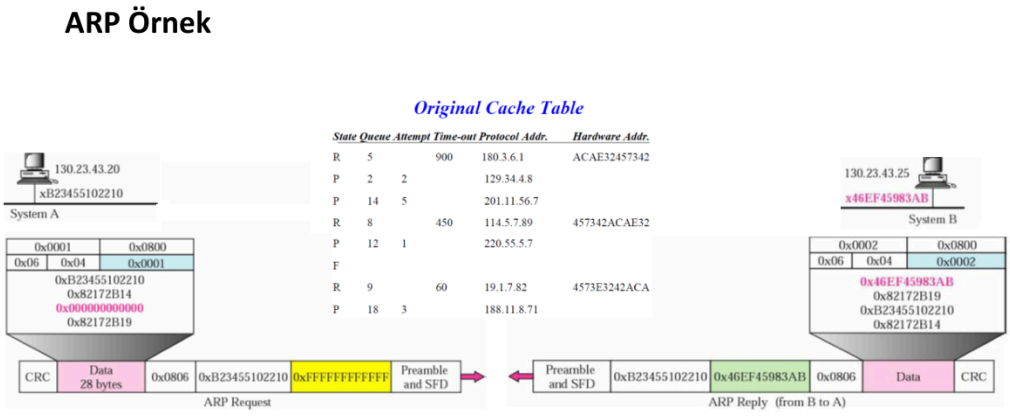
KRİTİK KAPSÜLLEME MANTIĞI: "14 Byte Ethernet + 28 Byte ARP = 42 Byte" Nereden Gelir?

- **Slayt 11 ve Slayt 12:** Şablonun genel adı **ARP Paketi**dir ve boyutu **28 Byte**'tır (`2B Hardware Type + 2B Protocol Type + 1B HW Len + 1B Proto Len + 2B Operation + 6B Sender MAC + 4B Sender IP + 6B Target MAC + 4B Target IP = 28 Byte`).
- Bu 28 byte'lık ARP şablonunun `Operation` (`packet[20..21]`) alanına `1` (`0x0001`) yazılırsa adı **ARP Request Paketi**, `2` (`0x0002`) yazılırsa adı **ARP Reply Paketi** olur.
- Slayt 11'de pembe oklarla gösterildiği gibi, bu **28 byte'lık ARP (Request veya Reply) paketi** doğrudan Ethernet çerçevesinin **`Data`** kısmına (`packet[14..41]`) yerleştirilir! (ARP paketinde IP başlığı yoktur).
- Başına **14 Byte Ethernet Başlığı** (`packet[0..13]`) eklendiğinde kabloya gönderilen toplam çerçeve **`14 + 28 = 42 Byte`** (`u_char packet[42]`) olur. Slayt 14'ün en altında bu 42 byte'lık çerçevenin tamamına da kısaca *ARP Request / ARP Reply* denmiştir.



Görsel: Hafta-2 Katman 2 PDF Sayfa 7, Slayt 13 – ARP Protokolünün 4 Kullanım Durumu (Case 1–4)

- **Slayt 13 (ARP 4 Kullanım Durumu — Sınav Senaryosu!):**
  - **Case 1 (Host → Aynı ağdaki Host):** ARP `Target IP` = IP datagramındaki `Destination IP`.
  - **Case 2 (Host → Başka ağdaki Host):** Paket önce yerel ağ geçidine (Router) gitmelidir; bu yüzden ARP `Target IP` = **Router'ın IP adresi** olur!
  - **Case 3 (Router → Başka ağdaki Router):** ARP `Target IP` = **Bir sonraki Router'ın IP adresi**.
  - **Case 4 (Router → Aynı ağdaki Hedef Host):** ARP `Target IP` = IP datagramındaki `Destination IP`.
- **Slayt 14 (ARP Cache Table Durum Harfleri):** `R` = *Resolved* (Çözümlenmiş, MAC biliniyor), `P` = *Pending* (Beklemede, istek atıldı), `F` = *Free* (Boş satır).



Görsel: Hafta-2 Katman 2 PDF Sayfa 7, Slayt 14 – ARP Önbellek Tablosu ve 28 Byte ARP Request / Reply Hex Örneği

3.4. Kod Analizi: 1-arpgonder.c (Satır 1–108)

`0-Katman2-3 Kodlar/1-arpgonder.c` dosyası, Slayt 12 ve Slayt 14'teki 42 Byte'lık ARP Request çerçevesini oluşturup gönderir:

Dizi İndeksi	Boyut	Katman ve Alan Adı	Kodda Atanan Değer	1-arpgonder.c Satır No ve Açıklama
`packet[0..5]`	6 Byte	Ethernet: Destination MAC	`0xff, 0xff, 0xff, 0xff, 0xff, 0xff`	**Satır 28–34:** Broadcast MAC adresi (Tüm LAN'a gider).
`packet[6..11]`	6 Byte	Ethernet: Source MAC	`0x54, 0x42, 0x49, 0x02, 0x31, 0x8c`	**Satır 38–44:** Gönderen cihazın MAC adresi.
`packet[12..13]`	2 Byte	Ethernet: Type (EtherType)	`0x08, 0x06` (`0x0806`)	**Satır 46–48:** Ethernet Data kısmında **ARP** olduğunu belirtir.
`packet[14..15]`	2 Byte	ARP: Hardware Type	`0x00, 0x01`	**Satır 53–55:** Donanım tipi `1` = Ethernet. *(Not: Satır 53 yorumundaki "=Request" ifadesi dizgi hatasıdır; Request kodu Satır 68'dedir!)*
`packet[16..17]`	2 Byte	ARP: Protocol Type	`0x08, 0x00` (`0x0800`)	**Satır 57–59:** Çözümlenen protokol tipi = IPv4.
`packet[18]`	1 Byte	ARP: Hardware Size	`0x06`	**Satır 61–62:** MAC adres uzunluğu = 6 byte.
`packet[19]`	1 Byte	ARP: Protocol Size	`0x04`	**Satır 64–65:** IPv4 adres uzunluğu = 4 byte.
`packet[20..21]`	2 Byte	ARP: Opcode (Operation)	`0x00, 0x01`	**Satır 67–69:** `1` = **ARP Request** (Cevap olsaydı `0x00, 0x02` olurdu).
`packet[22..27]`	6 Byte	ARP: Sender MAC	`0x54, 0x42, 0x49, 0x02, 0x31, 0x8c`	**Satır 71–77:** Gönderen MAC (`packet[6..11]` ile aynı).
`packet[28..31]`	4 Byte	ARP: Sender IP	`0x0a, 0x09, 0x1b, 0xe1`	**Satır 79–83:** Gönderen IP: `10.9.27.225` (Hex: `0a.09.1b.e1`).
`packet[32..37]`	6 Byte	ARP: Target MAC	`0x00, 0x00, 0x00, 0x00, 0x00, 0x00`	**Satır 85–91:** Hedef MAC henüz bilinmediği için `0` doldurulur (Slayt 12).
`packet[38..41]`	4 Byte	ARP: Target IP	`0x0a, 0x09, 0x1b, 0x64`	**Satır 93–97:** MAC adresi sorulan hedef IP: `10.9.27.100` (Hex: `0a.09.1b.64`).

Son olarak **Satır 101**'de `pcap_sendpacket(fp, packet, 42)` çağrısı ile 42 byte'lık dizi ağa gönderilir.

4. DERS (BÖLÜM 1): HAFTA 3 – KATMAN 3: IP, FRAGMANTASYON, OPSİYONLAR, CHECKSUM VE IPv6

Kaynak Dosya: Hafta-3 Katman 3 Protokolleri.pdf (PDF Sayfa 1–10, Slayt 1–20)

4.1. IPv4 Temel Özellikleri ve Başlık Yapısı (PDF Sayfa 2–3, Slayt 3–5)

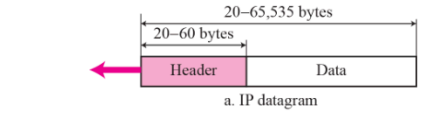
- **Slayt 4 (IPv4 Özellikleri):** Ağ katmanı paketlerine **datagram** denir. **Bağlantısız**, **en iyi teslim (best-effort)** ve **ortam bağımsız** bir protokoldür.
- **Boyut Kuralları (Slayt 4):** IP Başlığı **20–60 bayt** aralığındadır (Sabit kısım **20 bayt**, Opsiyonlar **0–40 bayt**). Toplam

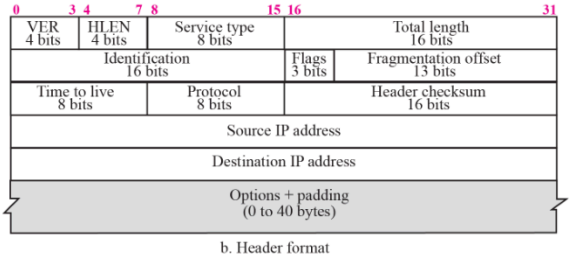
datagram boyutu **20–65,535 bayt** aralığındadır.

- **Slayt 5 (Protocol Alanı Değerleri Tablosu — `packet[23]`):** IP datagramının taşıdığı üst katman protokolünü belirtir: `1` = **ICMP**, `2` = **IGMP**, `6` = **TCP**, `17` = **UDP**, `89` = **OSPF**.

IP Protokolü (devam)

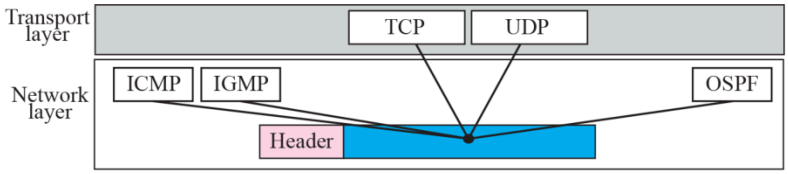
- Paketler datagramlar olarak adlandırılırlar.
- Datagram başlığı 20-60 bayt aralığındadır.
- Yönlendirme ve teslim ile ilgili bilgi saklar.
- Bağlantısız
- En iyi teslim
- Ortam bağımsız
- Kapsülleme var.





Görsel: Hafta-3 PDF Sayfa 2, Slayt 4 – IPv4 Datagram Başlık Formatı (20–60 Byte)

IP Datagramı Payload

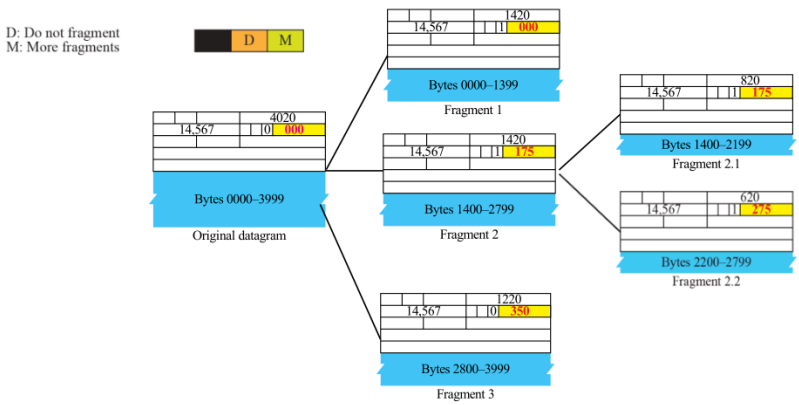


Value	Protocol	Value	Protocol
1	ICMP	17	UDP
2	IGMP	89	OSPF
6	TCP		

Görsel: Hafta-3 PDF Sayfa 3, Slayt 5 – IP Datagramı Payload ve Protocol Alanı Numaraları

4.2. IPv4 Fragmantasyon (Parçalama) ve Sayısal Hesaplama (PDF Sayfa 3–4, Slayt 6–7)

IP Datagramı Başlık Yapısı (Detaylı frag.)

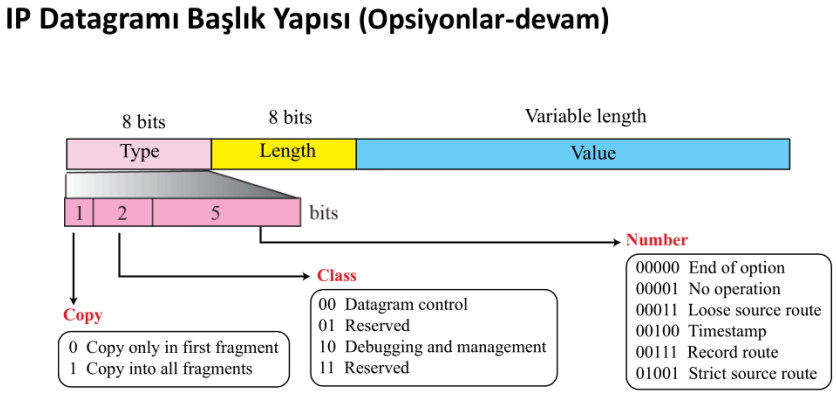


Görsel: Hafta-3 PDF Sayfa 4, Slayt 7 – IP Datagramı Detaylı Fragmantasyon Örneği

FRAGMANTASYON SAYISAL ÇÖZÜM FORMÜLLERİ (Slayt 7):

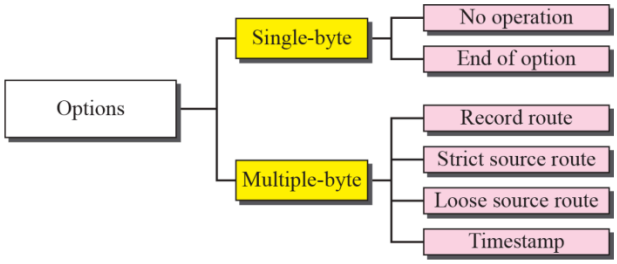
- **Bayraklar (3 Bit):** `D (Do not fragment) = 1` ise parçalanamaz. `M (More fragments) = 1` ise arkadan başka parçalar gelecek, `M = 0` ise bu son parçadır.
- **Total Length (Toplam Uzunluk):** Her parçaya 20 Byte IP başlığı eklendiği için `Total Length = Parça Veri Boyutu + 20`.
- **Fragmentation Offset:** Son parça hariç her parçadaki veri boyutu **8'in katı** olmalıdır. `Offset = İlk Veri Baytının Sıra Numarası / 8`.
- **Slayt 7 Örneğinin Adım Adım Hesabı (Orijinal Veri: 4000 Byte [0000–3999], Total Length: 4020, ID: 14,567):**
  - **Fragment 1 (Bytes 0000–1399, 1400B veri):** `Total Length = 1400 + 20 = 1420` | `M = 1` | `Offset = 0 / 8 = 000`.
  - **Fragment 2 (Bytes 1400–2799, 1400B veri):** `Total Length = 1400 + 20 = 1420` | `M = 1` | `Offset = 1400 / 8 = 175`.
    - *Fragment 2.1 (Bytes 1400–2199, 800B veri):* `Total Length = 800 + 20 = 820` | `M = 1` | `Offset = 1400 / 8 = 175`.
    - *Fragment 2.2 (Bytes 2200–2799, 600B veri):* `Total Length = 600 + 20 = 620` | **`M = 1`** (arkada hala Fragment 3 var!) | `Offset = 2200 / 8 = 275`.
  - **Fragment 3 (Bytes 2800–3999, 1200B veri):** `Total Length = 1200 + 20 = 1220` | **`M = 0`** (Son parça!) | `Offset = 2800 / 8 = 350`.

4.3. IPv4 Başlık Opsiyonları (Seçenekler) (PDF Sayfa 4–9, Slayt 8–17)



Görsel: Hafta-3 PDF Sayfa 5, Slayt 9 – Opsiyon Type Alanı (Copy, Class, Number)

IP Datagramı Başlık Yapısı (Opsiyonlar-devam)

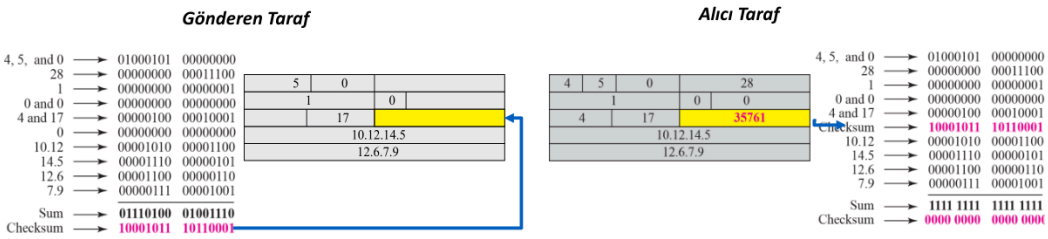


Görsel: Hafta-3 PDF Sayfa 5, Slayt 10 – Single-byte ve Multiple-byte Opsiyon Ağacı

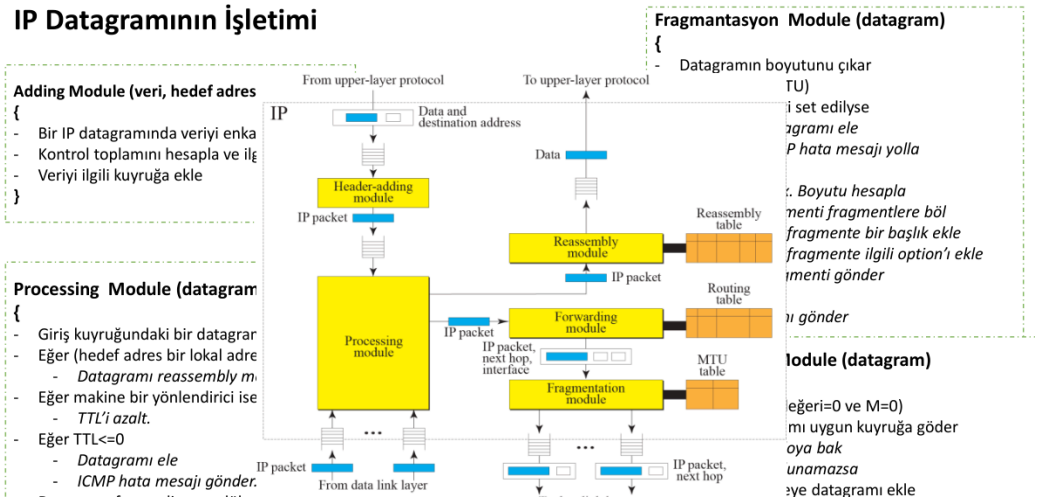
- **Slayt 8–9:** Sabit kısım 20 bayt, değişken kısım (opsiyonlar) **maksimum 40 bayt**tır. Opsiyon `Type` (8 bit) alanı: **`Copy` (1 bit:** `0` sadece ilk fragmente, `1` tüm fragmentlere kopyala) + **`Class` (2 bit:** `00` Datagram control, `10` Debugging/ management) + **`Number` (5 bit)**.
- **Single-byte Opsiyonlar (Slayt 10–11):**
  - **No operation (`Type: 1` / `00000001`):** Seçenekleri 16/32-bit sınırına **hizalamak (align)** için kullanılır.
  - **End of option (`Type: 0` / `00000000`):** Seçeneklerin sonunu **doldurmak (padding)** için kullanılır.
- **Multiple-byte Opsiyonlar (Slayt 11–17):**
  - **Record route (`Type: 7`):** Başlangıçta boş olan (maksimum **9 adreslik**) listeye, paketin geçtiği her yönlendirici kendi çıkış IP adresini yazar ve `Pointer` değerini 4 artırır (Slayt 11–12).
  - **Strict source route (`Type: 137`):** Gönderen, geçilecek **tüm** yönlendirici IP'lerini baştan belirler; paket liste dışındaki hiçbir yönlendiriciden geçemez (Slayt 13–14).
  - **Loose source route (`Type: 131`):** Listedeki yönlendiricilere uğramak zorunludur ancak aralarında başka yönlendiricilerden de geçebilir (Slayt 15).
  - **Timestamp (`Code: 68`):** Geçilen yönlendiricilerin IP adreslerini ve işlem zaman damgalarını kaydeder (Slayt 16–17).

4.4. IPv4 Kontrol Toplamı (Checksum), İşletim Modülleri ve IPv6 (PDF Sayfa 9–10, Slayt 18–20)

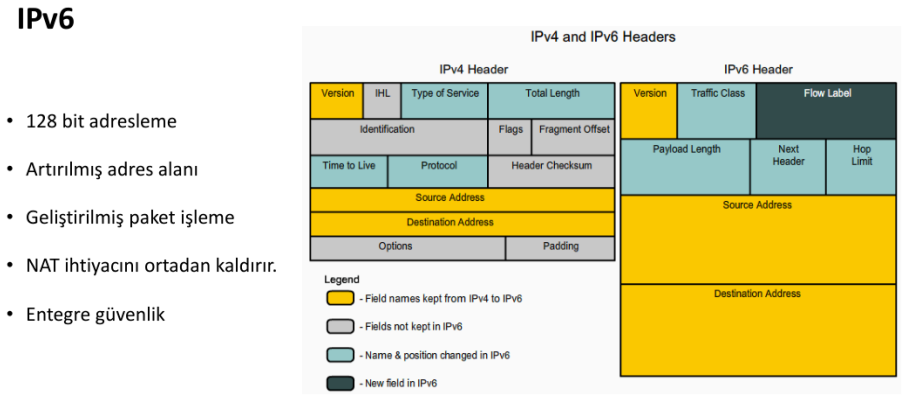
IP Datagramı Başlık Yapısı (Kontrol Toplamı)



Görsel: Hafta-3 PDF Sayfa 9, Slayt 18 – IP Başlığı Kontrol Toplamı (Gönderen ve Alıcı Taraf)



Görsel: Hafta-3 PDF Sayfa 10, Slayt 19 – IP Datagramının İşletimi (Adding, Processing, Fragmantasyon, Reassembly Modülleri)



Not: 4 milyar IPv4 adresi 4.000.000.000, 340 undesilyon IPv6 adresi

Görsel: Hafta-3 PDF Sayfa 10, Slayt 20 – IPv6 Özellikleri ve IPv4–IPv6 Başlık Karşılaştırması

- **Slayt 18 (Kontrol Toplamı / Checksum):** Başlık 16-bitlik satırlara bölünür. Gönderende Checksum alanı `0` yapılarak toplanır (`Sum`) ve **1'e tümleyeni (bitlerin tersi)** alınarak Checksum alanına (`35761`) yazılır. Alıcıda tüm başlık (Checksum dahil) toplandığında `Sum = 11111111 11111111` ve tersi alındığında **`Checksum = 0000 0000 0000 0000`** çıkmalıdır.
- **Slayt 19 (4 İşletim Modülü):**
  - **Processing Module:** Yönlendirici ise `TTL`'i 1 azaltır; `TTL <= 0` olursa datagramı eler ve **ICMP hata mesajı** gönderir.
  - **Fragmantasyon Module:** `Boyut > MTU` ise ve `D (Do not fragment)` biti set edilmişse datagramı eler ve **ICMP hata mesajı** gönderir; değilse parçalar.
  - **Reassembly Module:** `offset=0` ve `M=0` ise doğrudan kuyruğa gönderir; parçalıysa birleştirir. Süre aşımı (time-out) olursa tüm fragmentleri düşürür ve **ICMP hata mesajı** gönderir.
- **Slayt 20 (IPv6): 128 bit adresleme** (340 undesilyon adres), artırılmış adres alanı, geliştirilmiş paket işleme, **NAT ihtiyacını ortadan kaldırır**, entegre güvenlik. IPv6 başlığında `IHL`, `Identification`, `Flags`, `Fragment Offset` ve `Header Checksum` kaldırılmış; `TTL` → `Hop Limit`, `Protocol` → `Next Header`, `Type of Service` → `Traffic Class` olmuş ve **`Flow Label`** eklenmiştir.

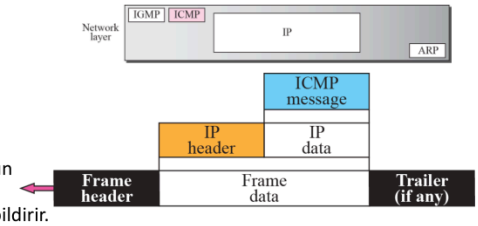
5. DERS (BÖLÜM 2): HAFTA 3 – ICMP PROTOKOLÜ VE KATMAN 3 C KODLARI

Kaynak Dosyalar: Hafta-3 Katman 3 Protokolleri.pdf (PDF Sayfa 11–16, Slayt 21–32) & 2-icmpgonder.c, ping_client.c, ping_server.c

5.1. ICMP Kapsüllemesi, 8 Byte Başlık ve Mesaj Kategorileri (PDF Sayfa 11–12, Slayt 21–23)

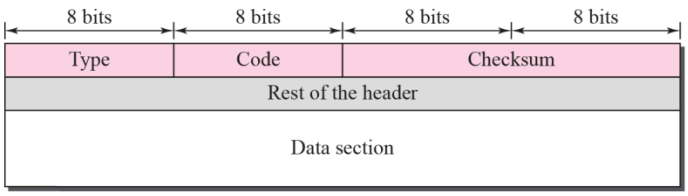
ICMP (Internet Control Message Protokol)

- ICMP iki amaçla kullanılır:
  - ➢ ***Hata raporlama iletileri***
  - ➢ ***Sorgu iletileri***
- Hata bildirim iletileri, bir yönlendiricinin veya ana bilgisayarın (hedefin) bir IP paketini işlerken karşılaşabileceği sorunları bildirir.
- Çiftler halinde oluşan sorgu iletileri, bir ana bilgisayarın veya ağ yöneticisinin bir yönlendiriciden veya başka bir ana bilgisayardan belirli bilgileri almasına yardımcı olur.
- Ayrıca, ana bilgisayarlar ağlarındaki yönlendiricileri keşfedebilir ve öğrenebilir ve yönlendiriciler bir düğümün iletilerini yönlendirmesine yardımcı olabilir.



Görsel: Hafta-3 PDF Sayfa 11, Slayt 21 – ICMP Mesajının IP Data ve Frame Data İçinde Kapsüllenmesi

ICMP



Category	*Type*	*Message*
Error-reporting messages	3	Destination unreachable
	4	Source quench
	11	Time exceeded
	12	Parameter problem
	5	Redirection
Query messages	8 or 0	Echo request or reply
	13 or 14	Timestamp request or reply

Görsel: Hafta-3 PDF Sayfa 11, Slayt 22 – ICMP 8 Byte Başlık Yapısı ve Hata / Sorgu Mesaj Kategorileri

- **Slayt 21 (Kapsülleme):** ARP'tan farklı olarak ICMP mesajı doğrudan Ethernet'e girmez; önce **IP datagramının** `IP data` **kısmına** (`Protocol = 1` ile), IP datagramı da **Ethernet çerçevesinin** `Frame data` **kısmına** kapsüllenir.
- **Slayt 22 (İki Ana Amaç):**
  1. **Hata Raporlama İletileri (Error-reporting):** `Type 3` (Destination unreachable), `Type 4` (Source quench), `Type 11` (Time exceeded), `Type 12` (Parameter problem), `Type 5` (Redirection).
  2. **Sorgu İletileri (Query):** `Type 8 veya 0` (Echo request / reply), `Type 13 veya 14` (Timestamp request / reply).

5.2. ICMP Hata Raporlama ve Sorgu Mesajları Kuralları (PDF Sayfa 12–16, Slayt 24–31)

ICMP – Hata raporlama mesajları

- ***Hedef ulaşılamaz mesaj formatı;***

Type: 3	Code: 0 to 15	Checksum
Unused (All 0s)		
Part of the received IP datagram including IP header plus the first 8 bytes of datagram data		

- 2 veya 3 kodlu hedefe ulaşılamayan mesajlar yalnızca hedef ana bilgisayar tarafından oluşturulabilir.
- Hedefe ulaşılamayan diğer iletiler yalnızca yönlendiriciler tarafından oluşturulabilir.
- Bir yönlendirici, paketin teslim edilmesini engelleyen tüm sorunları algılayamaz.
- IP protokolünde akış kontrol veya tıkanıklık kontrol mekanizması yoktur.

Görsel: Hafta-3 PDF Sayfa 12, Slayt 24 – Hedef Ulaşılamaz (Type 3) Formatı ve Kuralları

ICMP – Hata raporlama mesajları

- ***Yeniden yönlendirme mesaj formatı;***

Type: 5	Code: 0 to 3	Checksum
IP address of the target router		
Part of the received IP datagram including IP header plus the first 8 bytes of datagram data		

- Ana bilgisayar genellikle yavaş yavaş artırılan ve güncellenen küçük bir yönlendirme tablosu ile başlar. Bunu yapmanın araçlarından biri yönlendirme mesajıdır.
- Yönlendiriciden aynı yerel ağdaki bir ana bilgisayara bir yönlendirme mesajı gönderilir.



Görsel: Hafta-3 PDF Sayfa 14, Slayt 28 – Yeniden Yönlendirme (Type 5 Redirection) Mesajı ve İşleyişi

ICMP – Hata raporlama mesajları

- ***Echo-istek ve echo-cevap mesaj formatları;***

Type 8: Echo request
Type 0: Echo reply

Type: 8 or 0	Code: 0	Checksum
Identifier		Sequence number
Optional data Sent by the request message; repeated by the reply message		

- Bir ana bilgisayar veya yönlendirici tarafından bir echo isteği mesajı gönderilebilir.
- Bir echo isteği mesajı alan ana bilgisayar veya yönlendirici tarafından bir echo-cevap mesajı gönderilir.
- Echo isteği ve Echo yanıtı mesajları;
- IP protokolünün çalışmasını kontrol etmek için ağ yöneticileri tarafından kullanılabilir.
- Ana bilgisayarın ulaşılabilirliğini test edebilir (***ping , traceroute***)

Görsel: Hafta-3 PDF Sayfa 15, Slayt 29 – Echo-İstek (Type 8) ve Echo-Cevap (Type 0) Formatı

SINAVDA KESİN SORULABİLECEK ICMP HATA KURALLARI (Slayt 24–30):

- **Ortak Kural (Slayt 24–28):** Tüm ICMP hata mesajları, hataya sebep olan paketin **IP başlığını + orijinal verinin ilk 8 baytını** (`IP header plus the first 8 bytes of datagram data`) içerir.
- **1) Hedef Ulaşılamaz (`Type: 3, Code: 0..15` — Slayt 24):**
  - `Code 2` **(Protokol ulaşılamaz) ve** `Code 3` **(Port ulaşılamaz)** mesajları **YALNIZCA hedef ana bilgisayar (Host)** tarafından oluşturulabilir!
  - Diğer tüm `Type 3` iletileri **YALNIZCA yönlendiriciler (Router)** tarafından oluşturulabilir!
- **2) Kaynak Söndürme (`Type: 4, Code: 0` — Slayt 25):** Yönlendirici veya hedefteki tıkanıklık nedeniyle **atılan her datagram için** gönderilir; kaynağın gönderimi yavaşlatmasını ister.
- **3) Zaman Aşımı (`Type: 11, Code: 0 veya 1` — Slayt 26):**
  - `Code 0`**:** Yalnızca **yönlendiriciler (Router)** tarafından `TTL = 0` olduğunda gönderilir.
  - `Code 1`**:** Yalnızca **hedef ana bilgisayar (Host)** tarafından, parçalanmış datagramın tüm parçaları süresi içinde gelmediğinde gönderilir.
- **4) Parametre Problemi (`Type: 12, Code: 0 veya 1` — Slayt 27):** Yönlendirici veya hedef host tarafından oluşturulabilir; `Pointer` alanı hatanın olduğu baytı gösterir.
- **5) Yeniden Yönlendirme / Redirection (`Type: 5, Code: 0..3` — Slayt 28):** Yönlendiriciden (R1), **aynı yerel ağdaki (LAN)** bir ana bilgisayara (Host A) gönderilerek yönlendirme tablosunu hedef yönlendirici (R2) ile güncellemesini sağlar.
- **6) Sorgu Mesajları (Slayt 29–30):** `Type 8 / 0` (Echo Request / Reply — `ping, traceroute`) ve `Type 13 / 14` (Timestamp Request / Reply — **saatler senkronize olmasa bile** gidiş-dönüş süresi RTT hesabı ve saat eşitleme yapar).

5.3. Kod Analizi 1: 2-icmpgonder.c (Satır 1–118) — 50 Byte ICMP Echo Request

`0-Katman2-3 Kodlar/2-icmpgonder.c` dosyası, **14 Byte Ethernet Başlığı + 20 Byte IPv4 Başlığı + 16 Byte ICMP (8B Başlık + 8B Veri) = 50 Byte** (`u_char packet[50]`) uzunluğunda bir Ping isteği oluşturup gönderir:

Dizi İndeksi	Boyut	Katman ve Alan Adı	Kodda Atanan Değer	2-icmpgonder.c Satır No ve Açıklama
`packet[0..5]`	6 Byte	L2 Ethernet: Hedef MAC	`00:50:56:fc:18:27`	**Satır 24–29:** Alıcı MAC adresi.
`packet[6..11]`	6 Byte	L2 Ethernet: Kaynak MAC	`00:0c:29:cd:27:50`	**Satır 32–37:** Gönderen MAC adresi.
`packet[12..13]`	2 Byte	L2 Ethernet: Tip (EtherType)	`0x08, 0x00` (`0x0800`)	**Satır 40–41:** Üst katmanın **IPv4** olduğunu belirtir.
`packet[14]`	1 Byte	L3 IPv4: Version + HLEN	`0x45`	**Satır 45:** `4` = IPv4, `5` = `5 × 4 = 20 Byte` IP başlığı.
`packet[15]`	1 Byte	L3 IPv4: Service Type	`0x00`	**Satır 48:** Servis alanı (TOS).
`packet[16..17]`	2 Byte	L3 IPv4: Total Length	`0x00, 0x24` (`36 Byte`)	**Satır 51–52:** `20 Byte IP + 16 Byte ICMP = 36 Byte` (Hex `0x24`).
`packet[18..19]`	2 Byte	L3 IPv4: Identification	`0x10, 0xfd`	**Satır 55–56:** Paket kimlik numarası.
`packet[20..21]`	2 Byte	L3 IPv4: Flags + Offset	`0x00, 0x00`	**Satır 59–60:** Parçalanma yok.
`packet[22]`	1 Byte	L3 IPv4: TTL	`0x40` (`64`)	**Satır 63:** Yaşam süresi = 64. (`ping_client.c` Satır 182'de okunur!).
`packet[23]`	1 Byte	L3 IPv4: Protocol	`0x01` (`1 = ICMP`)	**Satır 66:** IP'nin içinde **ICMP** taşındığını belirtir (Slayt 5).

`packet[24..25]`	2 Byte	L3 IPv4: Header Checksum	`0x00, 0x00`	**Satır 69–70:** IP başlık kontrol toplamı.
`packet[26..29]`	4 Byte	L3 IPv4: Kaynak IP	`0xc0, 0xa8, 0x0b, 0x80`	**Satır 73–76:** `192.168.11.128`.
`packet[30..33]`	4 Byte	L3 IPv4: Hedef IP	`0xd8, 0x3a, 0xce, 0xc4`	**Satır 79–82:** `216.58.206.196`.
`packet[34]`	1 Byte	ICMP: Tip Alanı (Type)	`0x08`	**Satır 86:** `8` = **ICMP Echo Request** (Cevapta `0x00` yapılır).
`packet[35]`	1 Byte	ICMP: Kod Alanı (Code)	`0x00`	**Satır 89:** `0` (Not: `ping_client.c` Satır 295'te özel olarak `0x08` verilmiştir).
`packet[36..37]`	2 Byte	ICMP: Checksum	`0x00, 0x00`	**Satır 92–93:** ICMP kontrol toplamı.
`packet[38..41]`	4 Byte	ICMP: Rest of Header	`0x11, 0x8d, 0x00, 0x1b`	**Satır 96–99:** Identifier (2B) + Sequence Number (2B) (Slayt 29).
`packet[42..49]`	8 Byte	ICMP: Zaman Damgası (Data)	`0xa6, 0x5d, 0x4d, 0x5e...`	**Satır 102–109:** 8 byte isteğe bağlı veri / zaman damgası.

5.4. Kod Analizi 2: ping_server.c (Satır 1–176) ve ping_client.c (Satır 1–318)

- **1) Dinamik BPF Filtresi Oluşturma (`filter_ip_address` — `ping_server.c` Satır 148–176 & `ping_client.c` Satır 186–212):**
  - `pcap_findalldevs(&alldevs, error_buffer)` ile sistemdeki tüm ağ arayüzlerini bağlı liste olarak alır.
  - `strcmp(d->name, device) == 0` ile aktif cihazı bulur; `a->addr->sa_family == AF_INET` kontrolüyle IPv4 adresini `inet_ntoa(((struct sockaddr_in *)a->addr)->sin_addr)` üzerinden stringe çevirir.
  - `strcat(filter_exp, dest_ip)` ile `" icmp && dst <kendi_IP_adresi>"` filtresini oluşturur.
- **2) Sunucuda Echo Request Yakalayıp Echo Reply Gönderme (`ping_server.c` Satır 82–137):**
  - **Satır 78:** `pcap_loop(handle, total_packet_count, my_packet_handler, my_arguments)` (`total_packet_count = 0` olduğu için sonsuz döngüde dinler).
  - **Satır 94:** `if (packet[34] == 0x08 && packet[35] == 0x08)` ile gelen paketin Echo Request olduğunu kontrol eder ve `my_packet[50]` dizisine kopyalar.
  - **Satır 107–118 (Sınavda En Kritik Swap Bloğu):**

```
// ping_server.c Satır 107-118: Kaynak/Hedef MAC ve IP Adreslerini Takas Edip Echo Reply Gönderme
for (int i = 0; i < 4; i++) {
    swap(&my_packet[i + 0], &my_packet[i + 6]);   // MAC adresi ilk 4 byte takası (packet[0..3] <-> packet[6..9])
    swap(&my_packet[i + 26], &my_packet[i + 30]); // IP adresi 4 byte takası (packet[26..29] <-> packet[30..33])
}
swap(&my_packet[4], &my_packet[10]); // MAC 5. byte takası
swap(&my_packet[5], &my_packet[11]); // MAC 6. byte takası (Çünkü MAC 6 byte, IP ise 4 byte'tır!)

my_packet[34] = 0x00; // ICMP Type: Echo Request (0x08) -> Echo Reply (0x00) yapılır!
pcap_sendpacket(handle, my_packet, 50);
```

- **3) İstemcide IP Parçalama, Gönderim ve RTT Ölçümü (`ping_client.c` Satır 91–184):**
  - **Satır 94–121:** Komut satırından alınan hedef IP (`argv[1]`) ve yerel IP (`dest_ip_str`), `strtok(..., ".")` ve `atoi()` ile 4 elemanlı `dest_ip[4]` ve `src_ip[4]` dizilerine ayrılır ve `packet_generator(src_ip, dest_ip)` (Satır 214–318) çağrılır.
  - **Satır 130–149:** `while (s < atoi(argv[3]))` döngüsünde `pcap_sendpacket(fp, packet, 50)` ile paket atılır, `begin = clock()` başlatılır ve `pcap_loop(fp, 1, my_packet_handler, my_arguments)` ile 1 yanıt paketi beklenir.
  - **Satır 164–183:** `if (packet[34] == 0x00 && packet[35] == 0x08)` ile Echo Reply doğrulanır, `end = clock()` ile süre ölçülür ve `packet[30..33]` (IP), **`packet[22]` (TTL)** ve `time_spent` ekrana yazdırılır.